Kafrelsheikh University
Faculty of Engineering
Intelligent Systems Program

Multi-Objective Reinforcement Learning for Robust and Energy-Efficient Quadrupedal Locomotion

A graduation project thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering

Prepared by
Ahmad Ali Aamer
Ahmad Fouad Rashad
Mohammad Ayman Abo-Al-Nadar
Maryam Mohammad Rakha

Under the supervision of
Dr.Heba Gamal

Academic Year [2022 – 2027]

How to Use This Document

> This single document is both a writing guide and a fill-in template. Each section below contains:
>
> 1. A heading — keep it; this is the structure your thesis should follow.
> 2. *A grey **Guidance —** note explaining what to write and how long it should be. Delete these notes once you have written the section.*
> 3. A placeholder line in light grey where you type your own content.

Work top to bottom. Replace every bracketed placeholder and every grey note with your own writing. Before submission, make sure no grey guidance text or [bracketed] placeholder remains anywhere in the file. The general rules for writing style, formatting, figures, and referencing are collected at the end of this document — read them once before you start.

Table of Contents

Abstract

Guidance — *One paragraph, about 150–250 words, on a single page. Write it last. Cover four things in order: the problem you address, what you built or proposed, how you evaluated it, and the main result. No citations, no figures, no abbreviations that are not defined. A reader should understand your whole project from this paragraph alone.*

[Write your text here.]

Keywords:

Deep Reinforcement Learning, Proximal Policy Optimization (PPO), Quadrupedal Locomotion, Energy-Efficient Locomotion, Constrained Reinforcement Learning (CMDP), MuJoCo Simulation

Acknowledgements

We would like to express our sincere gratitude to **Dr. Heba Gamal** for her invaluable guidance, continuous encouragement, and constructive feedback throughout the completion of this research. Her expertise, support, and insightful suggestions greatly contributed to improving the quality of this work.

We also extend our sincere appreciation to the Faculty of Engineering and the Department of Artificial Intelligence for providing a supportive academic environment and the resources necessary to accomplish this project.

Finally, we would like to thank our families and colleagues for their continuous encouragement, patience, and support throughout this journey.

List of Figures, Tables, and Abbreviations

Guidance — *List figures and tables with their numbers and captions, and define every abbreviation you use. These lists can also be generated automatically by Word if you caption figures/tables consistently. Fill or remove as needed.*

Abbreviations

Abbreviation	Meaning
CNN	*Convolutional Neural Network*
[ABBR]	*[full meaning]*
[ABBR]	*[full meaning]*

Chapter 1: Introduction

1.1 Background and Motivation

Legged robots are capable of traversing uneven, cluttered, and unstructured terrain that is largely inaccessible to conventional wheeled or tracked platforms, making them well suited for applications such as inspection, search-and-rescue operations, and exploration of environments that are inaccessible to conventional mobile robots. Among legged designs, quadrupedal robots offer a practical balance between mechanical complexity and locomotion stability, and have consequently become a common platform for both industrial deployment and academic research.

Historically, quadrupedal locomotion has been addressed using model-based control techniques, including Model Predictive Control (MPC) and Central Pattern Generators (CPGs). While effective in structured settings, these approaches depend on precise dynamic models, accurate state estimation, and extensive manual tuning, which limits their adaptability when environmental conditions deviate from their design assumptions [1].

Deep Reinforcement Learning (DRL) has emerged as an alternative paradigm that learns locomotion policies directly through trial-and-error interaction, without requiring an explicit analytical model of the robot's dynamics. By formulating the locomotion task as a Markov Decision Process (MDP), a DRL agent learns to map sensory observations to low-level actuation commands so as to maximize a cumulative reward signal [2]. Relative to model-based control, DRL-based methods have demonstrated superior adaptability in complex and previously unseen environments [1].

However, adaptability alone is not sufficient for a locomotion policy to be considered successful. A practical controller must also remain robust to disturbances and uncertainty in its operating conditions, while keeping the energy cost of movement within reasonable bounds. Reward functions that emphasize velocity tracking and terrain traversal without explicitly accounting for energy expenditure tend to produce high-torque, high-impact gaits that are mechanically demanding and energetically wasteful [3]. Achieving energy efficiency instead typically requires incorporating explicit energy-related cost terms — such as control effort or mechanical power — into the learning objective, so that the resulting policy favors efficient rather than merely fast or superficially stable locomotion [3], [4]. Balancing these three concerns — locomotion performance, robustness, and energy efficiency — is therefore central to designing DRL-based controllers suitable for practical use.

Although previous studies have explored energy-aware locomotion, relatively few studies have systematically compared reward shaping and constraint-based optimization for improving energy efficiency while maintaining locomotion performance and robustness. This project investigates this comparison in a fully simulated setting, using the Unitree A1 quadruped model within the MuJoCo physics engine, with no physical hardware implementation involved. The findings of this work are intended to provide insight into the trade-offs between locomotion performance, robustness, and energy consumption under different optimization strategies.

1.2 Problem Statement

Deep Reinforcement Learning (DRL) has proven highly effective for synthesizing quadrupedal locomotion policies capable of adapting to varied and previously unseen terrain, largely because it removes the need for an explicit analytical model of the robot's dynamics [1], [2]. However, the effectiveness of a DRL-based controller is determined almost entirely by the design of its reward function, and the majority of reward formulations used in current locomotion research are structured primarily around task-level objectives such as tracking a commanded velocity or successfully traversing a given terrain [3]. When energy expenditure is not explicitly represented in the reward, the optimization process has no incentive to avoid mechanically inefficient behavior, and the resulting policies frequently exhibit high joint torques and abrupt, high-impact contact patterns even when they satisfy the primary task objective [3].

Optimizing locomotion performance in isolation is therefore insufficient for producing controllers suitable for practical use. A policy that tracks velocity commands accurately but does so through energetically wasteful motion offers limited value in contexts where power availability, actuator wear, or thermal limits are of concern. Because reward design inherently involves balancing multiple, often competing, objectives, an emphasis on task performance alone can lead to reward hacking, in which the agent satisfies the literal reward signal without acquiring the underlying, generally desirable behavior that the reward was meant to encourage [Wang et al., 2022]. This underscores the need to treat energy efficiency not as an incidental by-product of training, but as an explicit component of the optimization problem.

At the same time, energy efficiency cannot be pursued in isolation from robustness. A controller trained to minimize energy expenditure under a narrow set of conditions may fail to generalize once subjected to disturbances such as variable ground friction, changes in payload, or external perturbations. For an energy-efficient policy to be meaningful beyond the specific conditions under which it was trained, its performance must be evaluated — and ideally maintained — under such variability. This places locomotion performance, energy efficiency, and robustness in direct tension with one another, requiring an optimization approach capable of balancing all three rather than treating any one of them as secondary.

Two broad strategies exist for incorporating energy efficiency into the learning objective: soft-constraint approaches, which add energy-related penalty terms to a single scalarized reward function, and hard-constraint approaches, which formulate the problem as a Constrained Markov Decision Process (CMDP) and enforce energy budgets explicitly, for example through Lagrangian-based methods [Srisuchinnawong & Manoonpong, 2025], [Gangapurwala et al., 2020]. While both strategies appear in the literature, comparatively little work has directly and systematically compared their relative effectiveness at balancing energy efficiency against locomotion performance and robustness under a shared experimental setting.

This thesis addresses this problem by developing and evaluating both a soft-constraint (weighted-sum) reward formulation and a hard-constraint CMDP formulation for energy-efficient quadrupedal locomotion, entirely within a MuJoCo-based simulation environment using the Unitree A1 quadruped model. The aim is to determine whether explicitly constraining energy consumption yields a more favorable trade-off between

efficiency, task performance, and robustness than conventional reward shaping, under a consistent and reproducible simulation-only setting.

1.3 Objectives

1. To develop a Proximal Policy Optimization (PPO)-based locomotion framework for the Unitree A1 quadruped within the MuJoCo simulation environment, capable of achieving stable velocity tracking and gait stability as a baseline for subsequent energy-efficiency optimization.
2. To improve the energy efficiency of the learned locomotion policy by incorporating mechanical power and control effort ($\Sigma \tau^2$) into the reinforcement learning objective, and to compare a soft-constraint reward-shaping approach with a hard-constraint Constrained Markov Decision Process (CMDP) formulation, using Cost of Transport (CoT) as the primary evaluation metric.
3. To evaluate the robustness and overall locomotion performance of the resulting policies under domain randomization — including variable ground friction, mass uncertainty, and external push perturbations — and to analyze the trade-offs between robustness, energy efficiency, and locomotion performance.

1.4 Scope and Limitations

This thesis focuses on a simulation-based investigation of energy-efficient quadrupedal locomotion and does not involve any physical robot, hardware implementation, or real-world deployment. All development, training, and evaluation are conducted within the MuJoCo physics engine using the Unitree A1 quadruped model, and no sim-to-real transfer is attempted as part of this work.

The scope of the project covers three stages of development. First, a baseline locomotion policy is trained using Proximal Policy Optimization (PPO) with an objective focused solely on velocity tracking and gait stability. Second, energy-related metrics — mechanical power and control effort — are incorporated into the learning objective, and two optimization strategies for enforcing energy efficiency, namely soft-constraint reward shaping and a hard-constraint Constrained Markov Decision Process (CMDP) formulation, are implemented and compared using the Cost of Transport (CoT) metric. Third, the resulting policies are evaluated for robustness under domain randomization, limited to variations in ground friction, mass uncertainty, and external push perturbations.

Several aspects fall outside the scope of this thesis. The project does not address perception-based locomotion, such as policies that rely on visual or depth information, and considers only proprioceptive observations. It does not investigate advanced agility behaviors such as parkour, jumping over gaps, or climbing obstacles, nor does it explore hierarchical reinforcement learning, teacher-student distillation, or motion imitation techniques such as Adversarial Motion Priors. The

project is also restricted to the Unitree A1 platform and does not generalize its findings to other quadrupedal robots.

A key limitation of this work is that all results are obtained exclusively in simulation, and the extent to which the trained policies would transfer to physical hardware is not evaluated. Therefore, the findings should be interpreted within the assumptions and constraints of the adopted simulation environment.

1.5 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 1: Introduction presents the background and motivation for the project, states the problem addressed, defines the objectives, and outlines the scope and limitations of the work. Chapter 2: Background and Literature Review introduces the theoretical concepts underlying reinforcement learning-based quadrupedal locomotion and reviews related work relevant to energy-efficient and constrained locomotion policies, concluding with the identification of the research gap addressed by this thesis. Chapter 3: Proposed System and Methodology describes the overall system architecture, the simulation environment and task specification, and the methodology used to design and train the locomotion policies. Chapter 4: Implementation describes the implementation of details of the proposed methodology within the simulation environment. Chapter 5: Results and Discussion presents the experimental setup and evaluation of metrics, reports the results obtained, and discusses their implications. Chapter 6: Conclusion and Future Work summarizes the main findings of the thesis, states its limitations, and suggests directions for future work.

Chapter 2: Background and Literature Review

Guidance — *Around 5–8 pages. This chapter proves you understand the field and that your project fills a real gap. It is not a list of summaries — it is an argument that builds toward "and that is why our approach is needed." Every claim should be cited.*

2.1 Theoretical Background

2.1.1 Quadrupedal Locomotion

Quadrupedal locomotion refers to the coordinated control of a four-legged robotic system to achieve stable and adaptive movement across varied terrain [1]. Controlling such a platform is a fundamentally difficult problem in robotics. The control task involves high-dimensional state spaces. It also involves dynamics that are both non-linear and non-convex. In addition, the robot must contend with contact events that occur stochastically and are difficult to predict in advance [1], [2]. Unlike wheeled or tracked systems, a quadruped robot must continuously coordinate multiple limbs through closed kinematic chains. In this arrangement, several joints act together to determine the position of a single foot. The robot must also manage underactuated dynamics, meaning the number of actuated joints is insufficient to directly control every degree of freedom of the robot's body. This requires the controller to reason not only about individual joint commands, but also about the sequence in which each foot makes and breaks contact with the ground. An incorrect contact sequence can quickly destabilize the robot.

Biological quadrupeds solve an analogous problem through a combination of two sensing mechanisms. The first is proprioceptive feedback, which conveys internal information about limb position and force. The second is anticipatory exteroception, which allows the animal to perceive and prepare for upcoming terrain features before contact occurs [1]. This fusion of internal and external sensing enables agile and adaptive gaits, even on irregular or unpredictable surfaces. Replicating this level of coordination and balance maintenance in an artificial system has traditionally relied on hand-derived analytical models of the robot's dynamics. As the complexity of the target behaviour increases, however, such models become increasingly difficult to construct and tune. This motivates the use of learning-based approaches that can acquire locomotion behaviour directly from experience, rather than from an explicit mathematical description of the robot [1], [2].

2.1.2 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) addresses the limitations of purely analytical control by combining the sequential decision-making framework of Reinforcement Learning (RL) with the representational power of deep neural networks [3], [4]. In this framework, the control problem is formulated as a Markov Decision Process (MDP), defined by the tuple shown below [3], [4]:

(S, A, P, R, γ) ... (2.1)

Here, S is the set of possible states of the robot, and A is the set of available actions. P describes the probability of transitioning between states given an action, R is the reward received after each transition, and γ is a discount factor that determines the relative importance of future rewards.

The learning agent, referred to as the policy, interacts with this environment over time. It is trained to select actions that maximise the expected cumulative discounted reward, expressed as the objective shown below [3]:

$$J(\theta) = E[\tau \sim \pi\theta] \left[\Sigma\ (t=0 \text{ to } T)\ \gamma^t\ r\square \right] \quad \text{... (2.2)}$$

In this expression, θ represents the parameters of the policy, and τ denotes a trajectory generated by following that policy. The term $r\square$ is the reward received at time step t, and the summation accumulates these rewards along the trajectory while discounting those received further in the future by the factor γ^t.

Within this framework, deep neural networks serve as universal function approximators. Rather than requiring an explicit, hand-crafted rigid-body model of the robot's dynamics, the neural network can process high-dimensional, continuous state observations, such as joint angles and joint velocities, and map them directly to low-level motor actions [3]. DRL algorithms are broadly divided into two categories. Model-free approaches learn a policy or value function purely from interaction data. Model-based approaches additionally learn or exploit a model of the environment's dynamics [4]. A widely used structure within model-free DRL is the actor-critic architecture [3], [4]. In this architecture, one network, the actor, is responsible for selecting actions, while a second network, the critic, estimates the value of states or actions to guide the actor's updates. This architecture provides the foundation for many modern policy-gradient algorithms used in continuous control tasks such as legged locomotion.

2.1.3 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is an on-policy, actor-critic-based policy gradient algorithm that has become a standard choice for continuous control problems, including legged locomotion [2], [5]. PPO was developed as a simplification of Trust Region Policy Optimization (TRPO). It retains TRPO's central goal of constraining how much the policy is allowed to change during a single update. However, it replaces TRPO's more computationally demanding constrained optimization procedure with a clipped surrogate objective function, which is simpler to implement while preserving training stability [2], [5].

This clipped objective is expressed as shown below [2]:

$$L^{CLIP}(\theta) = E[\ \min(\ r\square(\theta)\ \hat{A}\square\ ,\ \text{clip}(r\square(\theta),\ 1-\varepsilon,\ 1+\varepsilon)\ \hat{A}\square\ )\ ] \quad \text{... (2.3)}$$

Here, $r\square(\theta)$ is the probability ratio between the action probability under the updated policy and under the previous policy. The term $\hat{A}\square$ is the advantage estimate, which indicates whether the action taken was better or worse than expected under the

current policy. The clipping operation restricts $r_t(\theta)$ to remain within a small trust region around 1, bounded by a parameter ε, typically set to 0.2.

By taking the minimum between the unclipped and clipped versions of this term, PPO discourages excessively large policy updates. This design prevents the sudden performance collapse that can occur when a policy is updated too aggressively. Instead, it promotes more stable, monotonic improvement over training [2]. A recognised limitation of PPO is its on-policy nature. Because updates rely on data collected under the current policy, previously collected experience cannot be reused once the policy has changed. This results in relatively low sample efficiency compared to off-policy alternatives [5]. Despite this drawback, PPO's stability and relative simplicity of implementation make it a widely adopted algorithm for training locomotion policies in simulated physics environments.

While PPO provides a stable and reliable mechanism for optimizing locomotion policies, it does not, on its own, guarantee that the resulting behaviour satisfies additional requirements such as energy efficiency or robustness to disturbances. Enforcing such requirements calls for extending the standard reinforcement learning formulation to explicitly account for constraints alongside the primary reward objective. Consequently, although PPO provides a stable optimization framework, additional mechanisms are required to explicitly enforce safety, robustness, and energy-related constraints. The following sections therefore introduce Constrained Markov Decision Processes (CMDPs) and Constrained Reinforcement Learning (CRL), which extend the conventional reinforcement learning framework by incorporating such constraints into the optimization process.

2.2 Related Work

Guidance — *Review the most relevant existing approaches. Group them by theme or method, not one paragraph per paper. For each group, say what was done, what worked, and what its weakness is. Compare approaches rather than listing them.*

[Write your text here.]

Guidance — *A comparison table of related work is strongly recommended. Replace the example below with your own.*

Reference	Approach and limitation
[Author, Year]	*[method used — strength — limitation relevant to your problem]*
[Author, Year]	*[method used — strength — limitation relevant to your problem]*

2.3 Research Gap

Guidance — *Close the chapter by stating clearly what current work fails to do — the gap your project addresses. This sentence should connect directly to your problem statement in Chapter 1.*

[Write your text here.]

Chapter 3: Proposed System and Methodology

Guidance *— Around 6–10 pages and the technical heart of the thesis. Describe what you designed and the method you followed, in enough detail that a competent reader could reproduce it. Use the past tense and a neutral, objective voice.*

3.1 System Overview / Architecture

The proposed system was designed to learn energy-efficient locomotion policies for a quadrupedal robot in simulation. The Unitree A1 quadruped model was deployed within the MuJoCo physics engine, and the system was implemented and evaluated entirely within this simulated environment.

At a high level, the system followed a closed-loop reinforcement learning architecture. A velocity command was issued to the agent as a task specification. The simulated robot's state was then converted into an observation vector, which was passed to a PPO policy network responsible for generating joint-level actions. These actions were applied within the MuJoCo simulation environment, which advanced the robot's dynamics and produced the resulting state transition. A reward or cost signal was computed from this transition, incorporating both locomotion performance and energy-related terms. This signal was used to update the policy network through the PPO algorithm, and the resulting policies were subsequently evaluated to assess locomotion quality and energy efficiency.

Three training configurations were considered in this work: a baseline PPO formulation, PPO with reward shaping, and PPO with CMDP constraints. These configurations shared the same simulation environment and policy architecture, differing only in the strategy used to incorporate energy efficiency into the learning process.

[Insert System Architecture Diagram Here]

Figure 3.1 — Overall system architecture.

The remainder of this section describes each block of the architecture shown in Figure 3.1, in the order in which information flowed through the system during training.

Velocity Command. The velocity command represented the desired locomotion task given to the agent, specifying the target motion the policy was expected to track. It served as the reference signal against which the robot's actual motion was later compared when computing the locomotion component of the reward.

Observation Space. The observation space defined the information made available to the policy network at each control step. It was derived from the internal state of the simulated robot within MuJoCo and was constructed to give the policy sufficient information to relate its actions to the locomotion task and to the robot's physical condition. The specific variables included in the observation vector and their role in the learning process are described in Section 3.3.

PPO Policy Network. The PPO policy network mapped the observation space to a distribution over actions, from which joint-level control commands were sampled or selected. As an actor-critic method, PPO relied on this policy network together with a value estimator to guide learning. The specific architecture of the network is described in Section 3.3.

MuJoCo Simulation Environment. The MuJoCo simulation environment hosted the Unitree A1 model and was responsible for advancing the robot's rigid-body dynamics in response to the actions issued by the policy network. It computed the physical consequences of applied joint torques, including contact interactions between the robot's feet and the ground, and produced the subsequent state of the robot used to form the next observation.

Reward / Cost Computation. Following each simulation step, a reward or cost signal was computed from the resulting state transition. This computation combined a locomotion-tracking component, reflecting how closely the robot followed the velocity command, with an energy-related component. The precise formulation of this signal differed across the three configurations under comparison: it was expressed as a single scalarized reward in the baseline and reward-shaping configurations, and as a primary reward subject to an explicit constraint in the CMDP configuration. The detailed mathematical formulation of the reward functions and constraint terms is presented in Section 3.3.

Policy Update. The reward or cost signal, together with the trajectories collected during interaction with the simulation environment, was used to update the parameters of the PPO policy network. The update procedure followed the PPO algorithm described in Chapter 2, adjusting the policy while constraining the magnitude of change between successive updates to preserve training stability.

Performance Evaluation. Once training was completed, the resulting policies were evaluated to assess both locomotion performance and energy efficiency. This evaluation stage was used to compare the baseline PPO, reward-shaping, and CMDP configurations against one another. The evaluation metrics and experimental procedures used to compare the three training configurations are presented in Chapter 5.

3.2 Dataset

Guidance — *Describe the data: its source, size, format, classes/labels, and how it was split into training, validation, and test sets. State any pre-processing, cleaning, augmentation, or balancing you applied. If you collected your own data, explain how. Be honest about data quality and any missing values.*

[Write your text here.]

3.3 Methodology / Model Design

Guidance — *Explain your approach step by step: the model or algorithm, its components, the loss function, and the key design choices. Justify each major decision — why this model,*

why these hyper-parameters, why this metric. Include equations with numbers (Eq. 3.1, 3.2…).

[Write your text here.]

3.4 Tools and Technologies

***Guidance —** List the languages, frameworks, libraries, and hardware you used (e.g. Python, TensorFlow/PyTorch, GPU model). A short bulleted list is fine.*

- [Programming language and version]
- [Main frameworks / libraries]
- [Hardware / environment]

Chapter 4: Implementation

Guidance — *Around 4–7 pages. Describe how the design from Chapter 3 became a working system. Focus on the interesting and non-obvious parts — not every line of code. Use short, well-captioned code snippets only where they add real insight; put long code in an appendix.*

4.1 Implementation Details

Guidance — *Walk through the main modules and how they fit together. Explain the data pipeline, training procedure, and any engineering challenges you solved and how.*

[Write your text here.]

4.2 User Interface / Deployment

Guidance — *If your project has an interface, an API, or a deployment, describe it here and include screenshots. If not, you can remove this section.*

[Insert screenshots here, captioned Figure 4.x. Remove this section if not applicable.]

Chapter 5: Results and Discussion

Guidance — *Around 5–8 pages. Present what you found, then interpret it. Keep "results" (the numbers and figures) and "discussion" (what they mean) clearly separated. Do not hide weak results — explaining them well shows maturity.*

5.1 Experimental Setup and Metrics

Guidance — *State exactly how you evaluated: the metrics (accuracy, F1, RMSE, latency, etc.), the test data, and the baselines you compared against. Define each metric briefly so the reader can interpret the numbers.*

[Write your text here.]

5.2 Results

Guidance — *Present results in tables and figures, each with a number and caption. Refer to every table/figure in the text ("As shown in Table 5.1…"). Report numbers consistently — same units, same number of decimal places.*

[Insert results table here. Caption: Table 5.1 — Performance comparison.]

[Write your text here.]

5.3 Discussion

Guidance — *Interpret the results. Why did your method perform as it did? Compare with the related work from Chapter 2. Explain failures and surprises. Connect the findings back to your objectives in Chapter 1.*

[Write your text here.]

Chapter 6: Conclusion and Future Work

Guidance — *Around 2–3 pages. Close the loop. No new results here — only summary, reflection, and outlook.*

6.1 Conclusion

Guidance — *Restate the problem, what you did, and your main findings in a few short paragraphs. Make it clear that each objective from Chapter 1 was met.*

[Write your text here.]

6.2 Limitations

Guidance — *Honestly state what your solution does not do well and under what conditions it may fail. This is expected and respected in good engineering work.*

[Write your text here.]

6.3 Future Work

Guidance — *Suggest concrete next steps that would extend or improve the project. Be specific enough that another team could pick them up.*

- [Future direction 1]
- [Future direction 2]

References

Guidance — *List every source you cited, numbered in the order they first appear in the text, in IEEE style. Cite in the body using bracketed numbers like [1], [2]. Only include sources you actually cited. The entries below are format examples — replace them with your own.*

1. A. Author and B. Author, “Title of the paper,” Journal Name, vol. X, no. Y, pp. 1–10, Year.
2. C. Author, “Title of the conference paper,” in Proc. Conference Name, City, Year, pp. 1–8.
3. D. Author, Title of the Book, Xth ed. City: Publisher, Year.
4. E. Author. “Title of the web page.” Website Name. URL (accessed Month Day, Year).

Appendices

Guidance — *Use appendices for material that supports the thesis but would interrupt the flow: long code listings, full result tables, extra figures, datasheets, or survey forms. Label them Appendix A, Appendix B, and refer to them from the main text.*

Appendix A: [Title]

[Write your text here.]

General Guidelines (Read Once Before You Start)

The rules below apply across the whole thesis. They are reference material — they are not part of your final submission, so delete this whole section before you hand in.

Writing Style

- Write in clear, formal English. Short sentences beat long ones. Say one thing per sentence.
- Use the past tense for what you did (“we trained the model”) and the present tense for facts that stay true (“the ReLU function returns…”).
- Be consistent: use the same term for the same thing throughout. Do not switch between “model,” “network,” and “system” for the same object.
- Define every abbreviation the first time you use it, then use the short form.
- Avoid unsupported claims. If you state something as fact, either cite a source or show your own evidence for it.
- Do not copy text from sources. Paraphrase in your own words and cite. Plagiarism is the fastest way to fail.

Formatting

Item	Recommended setting
Page size	*A4, portrait*
Margins	*2.5 cm (1 inch) on all sides*
Body font	*Arial or Times New Roman, 11–12 pt*
Line spacing	*1.5 lines in body text*
Headings	*Bold, numbered (1, 1.1, 1.1.1)*
Alignment	*Justified body text*
Page numbers	*Bottom of every page*
Captions	*Below figures, above tables, numbered per chapter*

Figures, Tables, and Equations

- Number figures and tables per chapter: Figure 3.1, Table 5.2, and so on.
- Every figure and table must be referred to in the text before it appears, and must have a caption.
- Make sure text inside figures is readable when printed. Redraw blurry screenshots.
- Number equations on the right margin, e.g. (3.1), and refer to them as “Eq. (3.1).”
- Never paste a figure you cannot explain. If it is in the thesis, you must be able to describe it.

Referencing (IEEE Style)

- Cite in the text with square brackets in order of appearance: “…as shown in [3].”
- Number the reference list in the order the citations first appear, not alphabetically.

- Use a reference manager (Zotero, Mendeley, or Word's built-in tool) to keep the list consistent.
- Prefer peer-reviewed papers, books, and official documentation over blogs and forums.

Guidance — *Format examples for common source types are shown in the References section above.*

Submission Checklist

Tick each item before you submit:

- □ Every bracketed [placeholder] and grey guidance note has been removed.
- □ Title page is complete with all names, IDs, supervisor, and date.
- □ Table of contents has been updated (correct headings and page numbers).
- □ Every figure and table is numbered, captioned, and referenced in the text.
- □ Every claim is either cited or supported by your own results.
- □ All references are in IEEE format and cited in the text in order.
- □ Spelling and grammar checked; consistent terms used throughout.
- □ Plagiarism / similarity check passed within the allowed limit.
- □ Approved by your supervisor before final printing.